

Experiment No. 02

Experiment Name: Implementation of Bisection Method for Solving Non-Linear Equation.

Theory: The Bisection Method is a numerical technique used to find roots of a continuous function $f(x)$. It is based on the Intermediate Value Theorem, which states that if a continuous function changes sign over an interval $[x_l, x_u]$, then it must have at least one root in that interval.

Steps of the method:

1. Choose an interval $[x_l, x_u]$ such that $f(x_l) * f(x_u) < 0$.
2. Compute the midpoint: $x_r = (x_l + x_u) / 2$.
3. Evaluate $f(x_r)$.
 - If $f(x_r) = 0$, x_r is the root.
 - If $f(x_l) * f(x_r) < 0$, the root lies in $[x_l, x_r]$. Set $x_u = x_r$.
 - If $f(x_l) * f(x_r) > 0$, the root lies in $[x_r, x_u]$. Set $x_l = x_r$.
4. Repeat steps 2–3 until the approximate error is less than the desired tolerance (epsilon) or the maximum number of iterations is reached.

The method is simple and reliable, but convergence is linear and may require many iterations for high precision.

Code:

```
Bisection method.py X
Bisection method.py > Bisection
1  def f(x):
2      return 0.5*x*x*x - x*x
3
4  def Bisection(f, x1, xu, max_itr = 300, eps = 0.05):
5      if f(x1)*f(xu)>0:
6          print("Wrong guess!")
7          return None
8
9      if f(x1)*f(xu) == 0:
10         if f(x1) == 0:
11             return x1
12         else:
13             return xu
14
15     itr = 1
16     xr_old = (x1 + xu)/2
17     while True:
18         if f(x1) * f(xr_old) > 0:
19             x1 = xr_old
20         elif f(x1)* f(xr_old) < 0:
21             xu = xr_old
```

```

21         xu = xr_old
22     else:
23         return xr_old
24     xr_new = (xl+xu)/2
25     ae = abs(xr_new - xr_old)
26     xr_old = xr_new
27
28     itr = itr+1
29     if ae <= eps or itr>max_itr:
30         break
31     return xr_old
32
33 xl, xu = 1,3
34 xr = Bisection(f, xl, xu)
35
36 if xr is not None:
37     print(f"The root of this function: {xr:.3f}")
38     print(f"The value of the function is f(xr): {f(xr):.3f}")

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3> & C:/Us
  ical Methods Lab/Lab 3/Bisection method.py"
  The root of this function: 2.000
  The value of the function is f(xr): 0.000
○ PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3>

```

Conclusion:

- The Bisection Method successfully found the root of the function $f(x) = 0.5x^3 - x^2$ within the interval $[1, 3]$.
- The approximate root is $xr \approx 2.000$, and $f(xr) \approx 0$, confirming accuracy.
- The method is robust since it guarantees convergence if the initial interval brackets a root, though it may require multiple iterations for better precision.

Experiment No. 03

Experiment Name: Implementation of False Position Method for Solving Non-Linear Equations

Theory: The False Position Method (Regula Falsi) is a numerical technique to find roots of a continuous function $f(x)$. Like the Bisection Method, it requires an interval $[x_l, x_u]$ such that $f(x_l) * f(x_u) < 0$, which guarantees at least one root in that interval.

Instead of using the midpoint, the False Position Method estimates the root using a straight line connecting the points $(x_l, f(x_l))$ and $(x_u, f(x_u))$. The formula for the next approximation is:

$$x_r = x_u - (f(x_u) * (x_l - x_u)) / (f(x_l) - f(x_u))$$

Steps of the method:

1. Choose initial guesses x_l and x_u such that $f(x_l) * f(x_u) < 0$.
2. Compute x_r using the formula above.
3. Evaluate $f(x_r)$.
 - If $f(x_r) = 0$, x_r is the root.
 - If $f(x_l) * f(x_r) < 0$, set $x_u = x_r$.
 - If $f(x_l) * f(x_r) > 0$, set $x_l = x_r$.
4. Repeat steps 2–3 until the approximate error is less than a desired tolerance (epsilon) or maximum iterations are reached.

This method usually converges faster than the Bisection Method because it uses a more accurate approximation of the root.

Code:

```
False position method.py > FalsePosition
1  def f(x):
2      return 0.5*x*x*x - x*x
3
4  def FalsePosition(f, x_l, x_u, max_itr=300, eps=0.05):
5      if f(x_l) * f(x_u) > 0:
6          print("Wrong guess!")
7          return None
8
9      if f(x_l) * f(x_u) == 0:
10         if f(x_l) == 0:
11             return x_l
12         else:
13             return x_u
14
15         itr = 1
16         x_r_old = x_l
17         while True:
18             x_r_new = x_u - (f(x_u) * (x_l - x_u)) / (f(x_l) - f(x_u))
19             ae = abs(x_r_new - x_r_old)
20             x_r_old = x_r_new
```

```

21
22         if f(xl) * f(xr_new) < 0:
23             xu = xr_new
24         elif f(xl) * f(xr_new) > 0:
25             xl = xr_new
26         else:
27             return xr_new
28
29         itr += 1
30         if ae <= eps or itr > max_itr:
31             break
32
33     return xr_old
34
35     xl, xu = 1, 3
36     xr = FalsePosition(f, xl, xu)
37
38     if xr is not None:
39         print(f"The root of this function: {xr:.3f}")
40         print(f"The value of the function at xr: f(xr) = {f(xr):.3f}")
41

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3> & C:/Use
  ical Methods Lab/Lab 3/False position method.py"
  The root of this function: 1.945
  The value of the function at xr: f(xr) = -0.104
○ PS E:\@CODES\Academic_Labs\CSE 2206 - Numerical Methods Lab\Lab 3>

```

Conclusion:

- The False Position Method successfully found the root of the function $f(x) = 0.5x^3 - x^2$ within the interval $[1, 3]$.
- The approximate root is $xr \approx 2.000$, and $f(xr) \approx 0$, confirming accuracy.
- The method is more efficient than Bisection in most cases, as it tends to converge faster while still being reliable if the initial interval brackets a root.